

face_identifier.py 파일 정리

1. 전체 코드

```
import os
from pathlib import Path

import cv2
import numpy as np
from insightface.app import FaceAnalysis

class FaceIdentifier:
    def __init__(
        self,
        known_face_dir="known_faces",
        face_sim_threshold=0.20,
        save_debug_crops=True,
        debug_crop_dir="outputs/face_crops",
        use_cache=True,
        face_region_top_ratio=0.40,
        face_region_side_margin_ratio=0.15,
    ):
        self.known_face_dir = known_face_dir
        self.face_sim_threshold = face_sim_threshold
        self.save_debug_crops = save_debug_crops
        self.debug_crop_dir = debug_crop_dir
        self.use_cache = use_cache
        self.face_region_top_ratio = face_region_top_ratio
        self.face_region_side_margin_ratio = face_region_side_margin_ratio

        self.face_cache = {}
        self.min_face_crop_size = 20

        if self.save_debug_crops:
            os.makedirs(self.debug_crop_dir, exist_ok=True)

        self.face_app = FaceAnalysis(
            name="buffalo_l",
            providers=["CPUExecutionProvider"],
        )
        self.face_app.prepare(ctx_id=-1, det_size=(640, 640))

        self.face_db = self._load_known_faces()

    def identify_from_path(self, person_id, image_path):
        try:
            full_body_crop = cv2.imread(str(image_path))

            if full_body_crop is None:
                raise ValueError(f"이미지를 읽을 수 없습니다: {image_path}")

            return self.identify_from_crop(person_id, full_body_crop)

        except Exception as e:
            return self._error_result(person_id, str(e))

    def identify_from_crop(self, person_id, full_body_crop):
        try:
            if self.use_cache and person_id in self.face_cache:
                return dict(self.face_cache[person_id])

            full_body_crop = self._ensure_bgr_image(full_body_crop)
            if full_body_crop is None:
```

```

        return self._no_face_result(person_id)

    face_crop = self._crop_face_region(full_body_crop)
    if face_crop is None:
        return self._no_face_result(person_id)

    face_crop_path = self._save_face_crop(person_id, face_crop)
    result = self._recognize_face(person_id, face_crop, face_crop_path)

    if self.use_cache and result["status"] == "recognized":
        self.face_cache[person_id] = dict(result)

    return result

except Exception as e:
    return self._error_result(person_id, str(e))

def _load_known_faces(self):
    face_db = {}
    base_dir = Path(self.known_face_dir)

    if not base_dir.exists():
        print(f"[경고] 등록 얼굴 폴더가 없습니다: {self.known_face_dir}")
        return face_db

    for person_dir in base_dir.iterdir():
        if not person_dir.is_dir():
            continue

        person_name = person_dir.name
        embeddings = []

        for image_path in person_dir.iterdir():
            if image_path.suffix.lower() not in [".jpg", ".jpeg", ".png", ".bmp"]:
                continue

            image = cv2.imread(str(image_path))
            if image is None:
                continue

            face = self._get_largest_face(image)
            if face is None:
                print(f"[등록 실패] {person_name}: 얼굴 검출 안됨 - {image_path.name}")
                continue

            embeddings.append(self._normalize_embedding(face.embedding))

        if embeddings:
            face_db[person_name] = embeddings
            print(f"[등록 완료] {person_name}: {len(embeddings)}장")
        else:
            print(f"[등록 실패] {person_name}: 사용 가능한 얼굴 이미지 없음")

    return face_db

def _recognize_face(self, person_id, face_crop, face_crop_path):
    if not self.face_db:
        return self._no_registered_faces_result(person_id, face_crop_path)

    face = self._get_largest_face(face_crop)

    if face is None:
        return self._no_face_result(person_id, face_crop_path)

```

```

embedding = self._normalize_embedding(face.embedding)
best_name = "Unknown"
best_score = -1.0

for person_name, embeddings in self.face_db.items():
    scores = [
        self._cosine_similarity(embedding, ref_embedding)
        for ref_embedding in embeddings
    ]
    score = max(scores)

    if score > best_score:
        best_score = score
        best_name = person_name

if best_score < self.face_sim_threshold:
    return {
        "person_id": person_id,
        "name": "Unknown",
        "score": round(float(best_score), 4),
        "status": "unknown",
        "face_crop_path": face_crop_path,
    }

return {
    "person_id": person_id,
    "name": best_name,
    "score": round(float(best_score), 4),
    "status": "recognized",
    "face_crop_path": face_crop_path,
}

def _crop_face_region(self, full_body_crop):
    height, width = full_body_crop.shape[:2]

    top_height = int(height * self.face_region_top_ratio)
    side_margin = int(width * self.face_region_side_margin_ratio)

    x1 = max(0, side_margin)
    x2 = min(width, width - side_margin)
    y1 = 0
    y2 = min(height, top_height)

    crop_width = x2 - x1
    crop_height = y2 - y1

    if crop_width < self.min_face_crop_size or crop_height < self.min_face_crop_size:
        return None

    face_crop = full_body_crop[y1:y2, x1:x2]

    if face_crop.size == 0:
        return None

    return face_crop

def _save_face_crop(self, person_id, face_crop):
    if not self.save_debug_crops:
        return None

    os.makedirs(self.debug_crop_dir, exist_ok=True)

    save_path = os.path.join(
        self.debug_crop_dir,
        f"person_{person_id}_face.jpg",
    )

```

```

if not cv2.imwrite(save_path, face_crop):
    raise RuntimeError(f"얼굴 후보 crop 저장 실패: {save_path}")

return save_path

def _get_largest_face(self, image):
    faces = self.face_app.get(image)

    if not faces:
        return None

    return max(
        faces,
        key=lambda face: (
            (face.bbox[2] - face.bbox[0])
            * (face.bbox[3] - face.bbox[1])
        ),
    )

def _ensure_bgr_image(self, image):
    if image is None or not isinstance(image, np.ndarray) or image.size == 0:
        return None

    if image.ndim == 2:
        return cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)

    if image.ndim != 3:
        raise ValueError("지원하지 않는 이미지 형식입니다.")

    if image.shape[2] == 3:
        return image

    if image.shape[2] == 4:
        return cv2.cvtColor(image, cv2.COLOR_BGRA2BGR)

    raise ValueError("지원하지 않는 이미지 채널 수입니다.")

@staticmethod
def _normalize_embedding(embedding):
    embedding = embedding.astype(np.float32)
    norm = np.linalg.norm(embedding) + 1e-8
    return embedding / norm

@staticmethod
def _cosine_similarity(a, b):
    return float(np.dot(a, b))

@staticmethod
def _no_face_result(person_id, face_crop_path=None):
    return {
        "person_id": person_id,
        "name": "Unknown",
        "score": -1.0,
        "status": "no_face",
        "face_crop_path": face_crop_path,
    }

@staticmethod
def _no_registered_faces_result(person_id, face_crop_path=None):
    return {
        "person_id": person_id,
        "name": "Unknown",
        "score": -1.0,
        "status": "no_registered_faces",
        "face_crop_path": face_crop_path,
    }

```

```
@staticmethod
def _error_result(person_id, error):
    return {
        "person_id": person_id,
        "name": "Unknown",
        "score": -1.0,
        "status": "error",
        "error": error,
    }
```

2. 클래스 역할

FacelIdentifier 클래스는 기존 프로젝트에서 저장된 전신 crop 이미지 또는 OpenCV 이미지 배열을 입력받아, known_faces 폴더에 등록된 얼굴과 비교하는 얼굴인식 후처리 모듈입니다.

이 클래스는 카메라를 열거나 YOLO 추적을 수행하지 않습니다. 영상 입력, 사람 추적, 전신 판단, crop 저장은 기존 모듈이 담당하고, 이 클래스는 전달받은 crop 이미지에 대해 등록 인물인지 판단하는 역할만 수행합니다.

- known_faces 폴더를 읽어 사람별 등록 얼굴 DB를 만듭니다.
- 전신 crop 상단부를 얼굴 후보 영역으로 잘라냅니다.
- InsightFace 로 얼굴 특징값을 추출하고 등록 얼굴 DB와 유사도를 비교합니다.
- 결과는 name, score, status, face_crop_path 를 담은 dict 형태로 반환합니다.
- 한 번 recognized 된 person_id 는 캐시에 저장해 같은 사람을 반복 계산하지 않도록 합니다.

3. 각 함수 역할

외부 파일에서 직접 사용할 함수는 identify_from_path()와 identify_from_crop()입니다. 나머지 함수들은 내부 처리를 나누기 위한 보조 함수입니다.

함수	역할
__init__()	옵션값 저장, debug crop 폴더 생성, InsightFace 로드, known_faces 얼굴 DB 생성
identify_from_path()	저장된 전신 crop 이미지 경로를 입력받아 얼굴 식별 수행
identify_from_crop()	OpenCV 이미지 배열을 직접 입력받아 얼굴 식별 수행
_load_known_faces()	known_faces 하위 폴더를 사람 이름으로 보고 등록 얼굴 임베딩 저장
_crop_face_region()	전신 crop 의 상단부와 중앙 영역을 잘라 얼굴 후보 crop 생성
_recognize_face()	얼굴 후보 crop 의 임베딩과 등록 얼굴 임베딩을 비교해 결과 결정
_get_largest_face()	여러 얼굴이 검출되면 가장 큰 얼굴 하나를 대표 얼굴로 선택
_save_face_crop()	디버깅용 얼굴 후보 crop 을 outputs/face_crops 폴더에 저장
_ensure_bgr_image()	입력 이미지를 OpenCV BGR 형식으로 보정
_normalize_embedding()	얼굴 특징 벡터를 정규화해 유사도 비교를 안정화
_cosine_similarity()	두 얼굴 임베딩의 코사인 유사도 계산

4. Input / Output 정리

이 모듈을 외부 파일에서 사용할 때는 person_id 와 전신 crop 정보를 넣으면 됩니다. 결과는 항상 dict 로 반환됩니다.

구분	이름	설명	예시
Input	person_id	PersonTracker 가 부여한 사람 추적 ID	1
Input	image_path	CropManager 가 저장한 전신 crop 이미지 경로	outputs/crops/person_1.jpg
Input	full_body_crop	OpenCV 에서 잘라낸 전신 crop 이미지 배열	frame[y1:y2, x1:x2]
Output	name	인식된 사람 이름. 등록 인물이 아니면 Unknown	father / Unknown
Output	score	등록 얼굴과의 유사도 점수	0.6401
Output	status	식별 결과 상태	recognized / unknown / no_face / error
Output	face_crop_path	디버깅용 얼굴 후보 crop 저장 경로	outputs/face_crops/person_1_face.jpg

status 값은 얼굴인식 결과를 구분하기 위한 상태값입니다.

status	의미	상황
recognized	등록 인물로 판단됨	score 가 face_sim_threshold 이상
unknown	등록 인물로 판단하지 않음	얼굴은 찾았지만 등록 얼굴과 충분히 비슷하지 않음
no_face	얼굴 검출 실패	전신 crop 상단부에 얼굴이 없거나 너무 작음
no_registered_faces	등록 얼굴 DB 없음	known_faces 가 비었거나 등록 사진에서 얼굴 검출 실패
error	예외 발생	이미지 경로 오류, 파일 읽기 실패 등

5. 외부 파일에서 사용하는 법

기존 프로젝트에서는 FullBodyChecker 가 FULL 로 판단한 뒤, CropManager 가 저장한 crop_path 를 FaceIdentifier 에 넘기면 됩니다.

```
from face_identifier import FaceIdentifier

identifier = FaceIdentifier(
    known_face_dir="known_faces",
    save_debug_crops=True,
)

result = identifier.identify_from_path(
    person_id=person_id,
    image_path=crop_path,
)

print(result)
```

여기서 person_id 는 PersonTracker 가 반환한 ID 이고, crop_path 는 CropManager 가 저장한 전신 crop 이미지 경로 입니다. image_path 대신 OpenCV 이미지 배열을 바로 넘기고 싶으면 identify_from_crop(person_id, full_body_crop)을 사용하면 됩니다.

6. 출력 예시

등록 인물로 판단된 경우입니다.

```
{
  "person_id": 1,
  "name": "father",
  "score": 0.6401,
  "status": "recognized",
  "face_crop_path": "outputs/face_crops/person_1_face.jpg"
}
```

얼굴은 찾았지만 등록 인물과 충분히 비슷하지 않은 경우입니다.

```
{
  "person_id": 1,
  "name": "Unknown",
  "score": 0.14,
  "status": "unknown",
  "face_crop_path": "outputs/face_crops/person_1_face.jpg"
}
```

전신 crop 상단부에서 얼굴을 찾지 못한 경우입니다.

```
{
  "person_id": 1,
  "name": "Unknown",
  "score": -1.0,
  "status": "no_face",
  "face_crop_path": "outputs/face_crops/person_1_face.jpg"
}
```

7. 클래스 전체 흐름도

```
FaceIdentifier 객체 생성
↓
known_faces 폴더에서 등록 얼굴 임베딩 DB 생성
↓
외부 파일에서 person_id 와 crop_path 또는 full_body_crop 입력
↓
전신 crop 상단부를 얼굴 후보 영역으로 crop
↓
InsightFace 로 얼굴 후보 crop 에서 얼굴 검출 및 임베딩 추출
↓
등록 얼굴 DB 와 코사인 유사도 비교
↓
score 가 기준값 이상이면 recognized
```

↓
기준값 미만이면 unknown
↓
얼굴이 없으면 no_face
↓
dict 형태로 결과 반환

8. 기존 프로젝트와 연결 위치

현재 단계에서는 기존 파일을 수정하지 않고 모듈만 분리해 두는 것이 안전합니다. 나중에 연결할 때는 전신 crop 이 저장된 직후 crop_path 를 FaceIdentifier 에 전달하면 됩니다.

단계	기존 모듈	연결 내용
1	PersonTracker	person_id, bbox 반환
2	FullBodyChecker	FULL 인 경우에만 얼굴인식 후보로 사용
3	CropManager	전신 crop 저장 후 crop_path 반환
4	FaceIdentifier	identify_from_path(person_id, crop_path) 호출
5	GUI / PersonStateManager	name, score, status 를 라벨이나 상태값에 반영

9. GPU / 연산 소모 점검

이 모듈만 기준으로 보면 GPU 를 직접 사용하는 부분은 없습니다. InsightFace 는 CpuExecutionProvider 로 설정되어 있고, MediaPipe, YOLO, VLM, torch, CudaExecutionProvider 를 사용하지 않습니다.

항목	사용 여부	설명
InsightFace	사용	CpuExecutionProvider 로 실행하여 GPU 사용을 최소화
MediaPipe	미사용	import 도 하지 않음. 전신 crop 상단부로 얼굴 후보 crop 생성
YOLO	미사용	PersonTracker 가 담당. 이 모듈은 crop 만 입력받음
VLM / Qwen	미사용	VLMWorker 와 독립적으로 동작
OpenCV	사용	이미지 읽기, crop 저장, 색상 채널 보정
NumPy	사용	임베딩 정규화, 코사인 유사도 계산

정리하면 face_identifier.py 는 저장된 전신 crop 한 장을 대상으로 CPU 기반 얼굴 식별을 수행하는 후처리 모듈입니다. GPU 부담을 늘리는 구조가 아니라, 기존 crop 결과를 재활용해 연산량을 줄이는 방향입니다.